

Probeklausur: Grundlagen der Programmierung

Kurs: RV-WWIBE124

Dozent: Phillip Goellner

Datum: _____

Allgemeine Hinweise:

- Die Bearbeitungszeit beträgt 120 Minuten.
- In Anbetracht der begrenzten Bearbeitungszeit sollten die Antworten kurz und faktenorientiert formuliert werden.
Logisch eindeutige und inhaltlich vollständige Stichpunkte sind erlaubt.
- Die Aufgaben dürfen direkt im dafür vorgesehenen Bereich auf dem Aufgabenblatt gelöst werden. Es ist aber auch erlaubt, die Aufgaben auf zusätzlichen Seiten zu beantworten.
- Es sind keinerlei Hilfsmittel (d.h. kein Skript, JavaDoc oder Taschenrechner) zugelassen.
- Die Benotung erfolgt entsprechend der Standard-Notenskala der DHBW Ravensburg.
- Seiten die keiner Matrikelnummer zugeordnet werden können, können nicht gewertet werden.
Daher bitte die Matrikelnummer auf jeder Seite oben rechts eintragen.
- Komplett unleserliche Schrift kann ebenfalls nicht gewertet werden.
Daher bitte leserlich schreiben.

Zusätzliche Hinweise zu den Programmier-Aufgaben:

- Generell ist davon auszugehen, dass in der entsprechenden Klasse keinerlei weitere Variablen, Konstanten, Konstruktoren oder Methoden definiert sind.
- Falls eine `run()` Methode vorgegeben ist, ist davon auszugehen, dass diese unmittelbar zum Start des Programms ausgeführt wird.
- Es ist nicht nötig, verwendete Klassen zu importieren.
- Sofern nicht explizit gefordert, ist ein Kommentieren des Codes nicht nötig.
- Bitte auf das korrekte Setzen von Klammern `() [] {}`, Semikola `;`, Dezimalpunkten `.` und Anführungszeichen `"` achten
- Bitte auf die korrekte Verwendung von Vergleichsoperatoren `< > =` achten.
Kombinierte Vergleichsoperatoren `≤ ≥` sind nicht erlaubt
- Bitte auf die korrekte Verwendung von arithmetischen Operatoren `+ - * / %` achten

Die Aufgaben beginnen auf der nächsten Seite.

Viel Erfolg!

Aufgabe 1: Theorie – 6 Punkte (kumuliert: 6 von 100)

Es folgt eine Tabelle mit Aussagen zu den Themen Exceptions und Vererbung. Kreuze jeweils an, ob die Aussage richtig oder falsch ist. [je 1 Punkt]

Aussage	Richtig	Falsch
Man behandelt Exceptions mit Try-Catch-Konstrukten.		
Unterklassen erben alle Attribute, Methoden und Konstruktoren von ihrer Oberklasse.		
Unterklassen von RuntimeException bezeichnet man als "Unchecked Exceptions"		
Dass eine Methode eine Exception werfen kann, wird mittels des Keywords throws in der Signatur deklariert.		
Klassen können von beliebig vielen Interfaces mittels des Keywords extends erben.		
Unterklassen können Methoden ihrer Oberklasse überschreiben. Dafür muss das Keyword overrides verwendet werden.		

Aufgabe 2: Angewandte Programmierung – 5 Punkte (kumuliert: 11 von 100)

Vervollständige die untenstehende Methode `run()`

- 1 Dekлариerte eine Variable und weise ihr den Ganzzahl-Wert 16 zu. [1 Punkt]
- 2 Danach prüfe ob die Variable einen größeren Wert als 11 hat.
Wenn dies der Fall ist, gebe einen beliebigen Text auf der Konsole aus. [3 Punkte]
- 3 Falls nicht, gebe den halben Variableninhalt auf der Konsole aus. [1 Punkt]

[illegible]

Code

```
void run() {
    System.out.print("A");
    startIgnition(2);
    liftOff();
    System.out.print("K");
}

void startIgnition(int numberOfTanks) {
    checkFuelTanks(numberOfTanks);
    igniteGas();
    System.out.print("E");
}

void checkFuelTanks(int numberOfTanks) {
    if (numberOfTanks > 2) {
        System.out.print("H");
    } else {
        System.out.print("F");
    }
}

int getDefaultTemperature() {
    System.out.print("C");
    return 1337;
}

void igniteGas(int temperature) {
    System.out.print("B");
}

void igniteGas() {
    igniteGas(getDefaultTemperature());
    System.out.print("D");
}

void liftOff() {
    System.out.print("L");
}
```

Aufgabe 8: Angewandte Programmierung – 7 Punkte (kumuliert: 42 von 100)

Erstelle die Methode `smallest()`, welche Ganzzahlen (a, b, c) als Übergabeparameter annimmt.

Es kann davon ausgegangen werden, dass immer drei unterschiedliche Werte an die Methode übergeben werden.

Die Methode soll immer den kleinsten Wert zurückgeben (als Rückgabewert).

Beispiele:

```
smallest(3, 2, 1)    →    1
```

```
smallest(-1, 7, 5)    →    -1
```

```
smallest(-7, -3, -1) → -7
```

Code

```
int smallest(int a, int b, int c) {
```

}

Aufgabe 10: Programmier-Verständnis – 5 Punkte (kumuliert: 55 von 100)

Gib jeweils an welcher Wert in die Variable a gespeichert wird. Beachte dabei die arithmetischen Regeln in Java; die Ausdrücke sind so zu verstehen, als wären sie Teil eines Java-Programmes (alle Zeilen sind kompilierbar). [je 1 Punkt]

Jede Zeile ist unabhängig zu betrachten.

Berechnung	Wert von a
Beispiel: <code>int a = 1 + 2;</code>	3
<code>double a = 0.5 * 3;</code>	
<code>int a = 3 / 4;</code>	
<code>double a = 7.0 / 2;</code>	
<code>int a = (int) 1.5 * 3;</code>	
<code>double a = 3 * (int) (2.5 + 2);</code>	

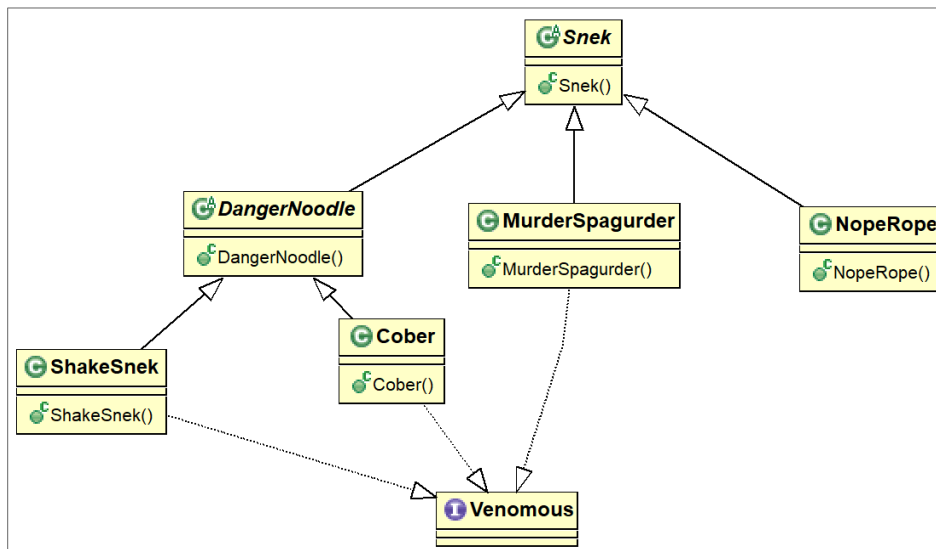
Aufgabe 11: Theorie und Programmier-Verständnis – 8 Punkte (kumuliert: 63 von 100)

Beantworte die folgenden Fragen im Kontext des untenstehenden Klassendiagramms.

- 1 Wie viele verschiedene Typen hat ein Objekt der Klasse `ShakeSnek` (ausgenommen `java.lang.Object`)? [1 Punkt]
- 2 Wie nennt man den Mechanismus, der ermöglicht, dass Objekte mehrere Typen haben? [1 Punkt]
- 3 In welcher Beziehung stehen die Klassen `Snek` und `MurderSpagurder`? [1 Punkt]
- 4 Bewerte ob die untenstehenden 5 Code-Blöcke (folgende Seite!) jeweils korrekt sind oder nicht. [je 1 Punkt]
Bitte betrachte jeden Block unabhängig von den anderen.

Bitte betrachte jeden Block unabhängig von den anderen.

Hinweise: `Snek` und `DangerNoodle` sind abstrakte Klassen. `Venomous` ist ein Interface.

[illegible]

Code

Aufgabe 13: Theorie – 6 Punkte (kumuliert: 77 von 100)

- 1 Nenne zwei Gründe, aus denen man Packages in der Java-Entwicklung verwenden sollte. [2 Punkte]
- 2 Was sind die Bestandteile eines Fully Qualified Class Names? [2 Punkte]
- 3 Was wird in Java im Stack gespeichert, was im Heap? [2 Punkte]

[illegible]

Aufgabe 14: Angewandte Programmierung – 4 Punkte (kumuliert: 81 von 100)

Entwickle die Methode `floor()`, welche eine positive Dezimalzahl (`double`) entgegen nimmt und als Ergebnis die abgerundete natürliche Zahl (`int`) zurückgibt.

Beispiele:

<code>floor(1.0)</code>	→ 1
<code>floor(3.2)</code>	→ 3
<code>floor(8.9)</code>	→ 8

Code

Aufgabe 17: Angewandte Programmierung – 10 Punkte (kumuliert: 100 von 100)

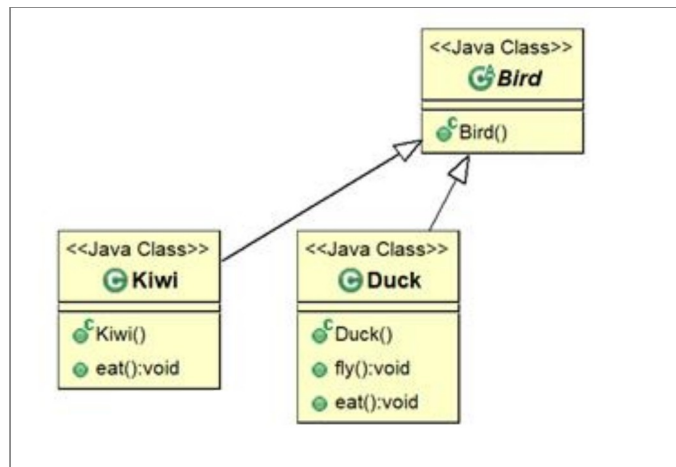
Vervollständige die untenstehende Klasse Zoo (folgende Seite!):

- 1 Zoo soll ein Attribut mit Namen `birds` haben, worauf nur innerhalb der Zoo-Klasse zugegriffen werden kann. `birds` soll den Datentypen Bird-Array haben und in der Lage sein 15 Bird-Objekte aufzunehmen (dafür darf direkt ein Wert zugewiesen werden). [4 Punkte]

Vervollständige die Methode `letAllDucksFly()`:

- 2 Iteriere mit einer Schleife über alle Inhalte des Bird-Arrays `birds`. [2 Punkte]
- 3 Überprüfe in jeder Iteration, ob das gegenwärtige Objekt von Typ Duck ist. [2 Punkte]
- 4 Falls das gegenwärtige Objekt von Typ Duck ist, rufe die Methode `fly()` des Objektes auf [2 Punkte]

Beachte bei den Implementierungen das Klassendiagramm!



Code

```
public class Zoo {
```

```
public void letAllDucksFly() {
```

}

}